Pattern Recognition 172 (2026) 112662



Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/pr





BAM: Backdoor defense based on adversarial mitigation

Run Lu [a,1], Peipeng Yu [a,1], Zhitao Huang [a], Zhihua Xia [a,*], Xiaoyan Jiang [b]

[a] Jinan University, Guangzhou, China
[b] China Unicom Digital Technology Co., Ltd, Beijing, China

ARTICLE INFO

Keywords:
Backdoor attack
Backdoor defense
Adversarial perturbation

ABSTRACT

Deep Neural Networks (DNNs) are vulnerable to backdoor attacks, where the model has good performance on normal data but behaves maliciously on samples with certain triggers. By introducing an input preprocessing module, it is possible to change the triggers contained in backdoored samples, making the modified triggers no longer match the hidden backdoor. Most existing input preprocessing-based defense works can effectively deal with visible triggers, but they often fail against invisible and robust triggers. In this paper, we propose a novel defense strategy named Backdoor Adversarial Mitigation (BAM) to mitigate the invisible and robust triggers by using adversarial perturbation. We first employ an untargeted adversarial attack algorithm to generate simulated backdoor samples. Subsequently, we combine these samples with benign samples to train an Backdoor Adversarial Perturbation Generator, which aims to mitigate backdoor triggers while minimally affecting the classification of benign samples. In the deployment phase, we further integrate image-blurring technique to weaken the visible triggers, enhancing the overall effectiveness of our method against both visible and invisible triggers. Extensive experiments on various backdoor attacks demonstrate the effectiveness of our method, outperforming state-of-the-art defense methods, even with a very small portion of clean training data.

1. Introduction

Deep neural networks (DNNs) have revolutionized a wide range of domains, including face recognition [1], autonomous driving [2], and medical diagnosis [3], among others. Due to constraints on data and computing resources, users often rely on third-party datasets, platforms, or pre-trained models to mitigate training costs. This practice also introduces potential security risks, with backdoor attacks being a prominent concern during the training stage. In a backdoor attack, the attacker manipulates the training data or process to subtly modify model parameters, creating a backdoor model. During the inference stage, as illustrated in Fig. 1, the backdoor model can exhibit malicious behavior when presented with inputs containing specific triggers, while behaving normally with benign samples. This concealment poses a significant challenge in defending against backdoor attacks, highlighting the need for effective defense strategies.

To mitigate the security threats posed by backdoor attacks, various defense methods have been developed, broadly categorized into detection-based defense, model repair-based defense, and input preprocessing-based defense. Detection-based defense focuses on identifying whether inputs contain the triggers or models contain backdoors, and then discarding backdoor models or samples [4], but those methods requires triggering the backdoor before detection, which reduces the efficiency during the model's deployment stage. Model repair-based methods involve retraining or modifying the user's model to eliminate backdoors [5], which imposes certain limitations on their applicability. In contrast, input preprocessing-based defense ensures the usability of both the backdoor model and backdoored samples without retraining or modifying the user's model. As depicted in Fig. 1, it introduces an input preprocessing module to mitigate triggers hidden in the input, rendering backdoors in the model ineffective [6]. However, as backdoor attack techniques continue to advance, backdoor triggers are becoming increasingly invisible and robust, thereby rendering many existing defense algorithms ineffective [7]. Therefore, effectively mitigating the effects of more invisible and robust triggers remains an unresolved issue.

In this paper, we propose a novel backdoor defense method based on adversarial mitigation, named Backdoor Adversarial Mitigation (BAM). The key idea of our approach lies in the ability of adversarial perturbations to substantially alter the classification of input samples with minimal modifications. Leveraging this principle, we employ adversarial perturbations to mitigate the features associated with backdoor triggers, rectifying the misclassification of backdoored samples.

* Corresponding author.
E-mail address: xia_zhihua@163.com (Z. Xia).
[1] These authors contributed equally to this work.

https://doi.org/10.1016/j.patcog.2025.112662
Received 7 April 2025; Received in revised form 26 October 2025; Accepted 28 October 2025
Available online 1 November 2025
0031-3203/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

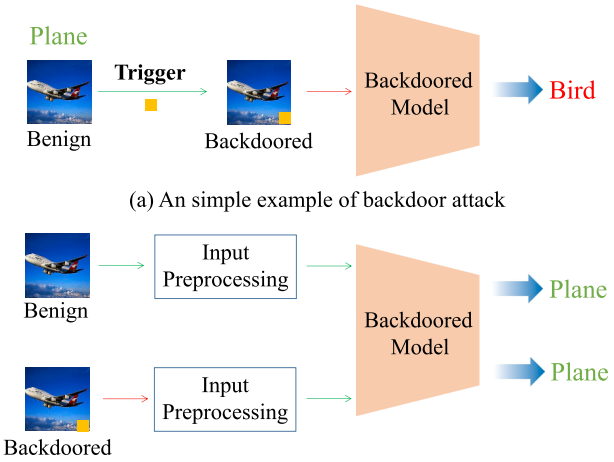


Fig. 1. Example of backdoor attack and illustrations of input preprocessing defense. Backdoor attacks have the potential to mislead the classification of target models, input preprocessing defense would mitigate the trigger effect and guide correct classification.

Specifically, since defenders cannot access information about the backdoor samples used by attackers, We first simulate backdoored samples to facilitate model training. As demonstrated in [5], adversarial attacks on backdoored models tend to produce adversarial examples with characteristics similar to backdoor samples,when adversarial attacks are performed on backdoored models, the neurons associated with the backdoor are more likely to be selected and activated during the generation of adversarial examples. This leads adversarial samples to share similar feature representations and behavioral patterns with true backdoor samples. Therefore, we utilize adversarial examples as simulated backdoored samples. Subsequently, we combine the simulated backdoor samples and benign samples to train an Backdoor Adversarial Perturbation Generator which aims to mitigate backdoor triggers while minimally affecting the classification of benign samples. During the deployment phase, we augment our strategy by incorporating image blurring to mitigate visible triggers, since blurring has minimal impact on the classification of benign samples compared to other data transform methods. Our contributions can be summarized as follows:

1) We introduce a novel Backdoor Adversarial Mitigation (BAM) scheme to defend invisble and robust backdoor attacks. We are the first to apply adversarial perturbation in input-preprocessing based backdoor defense, which does not require to modify the target model and are more efficient. At first, we employ untargeted adversarial attack to simulate backdoor samples from the target model. Then, we train an Backdoor Adversarial Perturbation Generator to generate perturbations for mitigating invisible triggers. Additionally, by incorporating blurring operation into our BAM, we enhance its capability to defend against both visible and invisible backdoor attacks.
2) Through extensive experiments and ablation studies, we substantiate the effectiveness of our proposed method. BAM outperforms well-known defense methods in mitigating various advanced backdoor attacks, while also demonstrating effectiveness with limited amounts of training data.

2. Background knowledge

In this section, we provide a concise overview of the background knowledge, including fundamental concepts of deep neural networks and backdoor attack, to establish the context for our study.

2.1. Deep neural networks

Deep Neural Networks (DNNs) have become a cornerstone of modern artificial intelligence due to their remarkable ability to model complex patterns and representations from large-scale data. A typical DNN consists of multiple layers, each applying a nonlinear transformation to its input. Formally, given an input $\mathbf{x}$, a DNN with L layers computes the output $\mathbf{y}$ as follows:

$$h^{(0)} = x, \quad h^{(l)} = f^{(l)}\left(W^{(l)}h^{(l-1)} + b^{(l)}\right), \quad l = 1, 2, \ldots, L, \tag{1}$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector of the lth layer, respectively, and $f^{(l)}(\cdot)$ denotes the nonlinear activation function. The final output $y = h^{(L)}$ is then used for prediction tasks.

The training process aims to find model parameters $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$ that minimize a loss function L over the training dataset $\{(x_i, y_i)\}_{i=1}^{N}$:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} L\left(f(x_i; \theta), y_i\right), \tag{2}$$

where $f(x_i; \theta)$ represents the network output for input x_i.

2.2. Backdoor attacks on neural networks

Backdoor attacks aim to manipulate the training process by injecting poisoned samples with a hidden trigger t, such that the model behaves normally on clean inputs but outputs an attacker-chosen target label y_t when the trigger is present. Formally, the poisoned training set D_p is constructed by adding triggered samples:

$$D_p = D \cup \{(x_i \oplus t, y_t) \mid (x_i, y_i) \in D_b\}, \tag{3}$$

where $\oplus$ denotes the operation of adding the trigger t to the benign input x_i, and $D_b \subset D$ is the subset of benign samples selected for poisoning.

During training on D_p, the model parameters are optimized to minimize the loss on both clean and triggered samples:

$$\min_{\theta} \frac{1}{|D_p|} \sum_{(x,y)\in D_p} L\left(f(x; \theta), y\right). \tag{4}$$

As a result, the trained model learns to associate the trigger t with the target label y_t, enabling the attacker to manipulate the model's output at inference time by embedding the trigger in inputs.

Understanding these mechanisms is critical for designing effective defenses against backdoor attacks, which include detection, removal, and robust training techniques.

3. Related work

In this section, we briefly introduce relevant literature encompassing mainstream backdoor attacks and defense methods, as well as adversarial attacks.

3.1. Backdoor attack

Backdoor attack refers to the embedding of a backdoor during the model training process, enabling the model to exhibit anomalous behavior when presented with samples containing triggers. Based on the trigger type, it can be categorized to two types: visible backdoor attacks and invisible backdoor attacks.

Visible Backdoor Attacks. The visible backdoor attacks involve embedding visible triggers on benign samples to implant backdoors. Gu et al. [8] introduced the first backdoor attack named BadNets, overlaying a local pixel block as trigger on benign sample. Liu et al. [9] designed a better trigger instead of simply using a pixel block, they learn the trigger from benign model, and then use the learned trigger to poison benign samples. Nguyen et al. [10] introduced a method that train a trigger generator to produce different triggers for various samples, but the triggers

are still visible. While such solutions are straightforward and effective, the visibility of the trigger renders them highly susceptible to detection and removal.

Invisible Backdoor Attacks. The invisible backdoor attacks use the triggers that indiscernible for attacking, the backdoored sample of those attacks are indistinguishable compared from its benign version, increasing the stealthiness of attacks. Chen et al. [11] first explored invisible backdoor attacks by blending patterns with certain transparency on benign samples as triggers. Nguyen et al. [12] proposed that use a specific small and smooth distortion fields to distort benign samples in a visually indistinguishable way, and the distorted samples used as poisoned samples for training. Li et al. [13] encoded an invisible string specified by the attacker into the sample using an encoder-decoder network. This encoded string served as the trigger, causing abnormal model behavior while meeting the invisibility and sample specificity. Jie et al. [14] extracted the edge structure of benign samples and filled them with poisonous ink information to generate the triggers, then embedded triggers into the benign samples using a deep injection network. Since the edge structure can maintain its semantics during data transformations, those triggers are more robust and also satisfies the invisibility and sample specificity. Tang et al. [7] developing a novel strategy that formulate trigger injection and perception under backdoor attack into message hiding and extraction under steganography, making it more challenging to defend against. Recently, backdoor attacks for other tasks have also made significant progress. Cheng et al. [15] employed a combination of double trigger poisoning, noise contrastive estimation loss, and a single trigger regularization strategy to improve the effectiveness of attacks in the visual object tracking task. Meanwhile, Wang et al. [16] introduced a novel poison node selection strategy that significantly improves the efficiency of graph-based backdoor attacks.

As backdoor attacks evolve, the triggers embedded in samples are increasingly moving towards invisibility and robustness, posing greater challenges for defense. This paper proposes a backdoor defense algorithm based on adversarial mitigation, aimed at mitigating invisible and robust backdoor triggers.

3.2. Backdoor defense

Based on the different strategies adopted by defenders, existing defense methods against backdoor attacks can be categorized into the following three types: 1)detection-based defense, 2)model repair-based defense, 3)input preprocessing-based defense.

Detection-based defense. Detection-based defense methods combat backdoor attacks by either identifying anomalous samples or detecting abnormal model. These methods can be categorized into defense based on input detection and defense based on model detection. Defense based on input detection focuses on detecting and filtering out backdoored samples from incoming inputs to prevent backdoor activation. Gao et al. [17] superimposed input samples onto a set of randomly selected normal samples and observed the model's output on the mixed images to determine whether a sample was a backdoor sample. Guo et al. [4] proposed that identify and filter malicious testing samples by analyzing their prediction consistency during the pixel-wise amplification process. In contrast, defense based on model detection aims to analyze the behavior or internal structure of the model to determine if it has been compromised, such as Xu et al. [18] trained multiple clean models and backdoor models to train a query set and a meta-classifier. The trained query set was input into the target model to extract features, which were then fed into the trained meta-classifier to determine whether the model was a backdoor model. Although detection-based methods have achieved good detection performance, they cannot guarantee the availability of backdoored samples or backdoor models, increasing the user's operational costs, and requires triggering the backdoor before detection which reduces the efficiency during the model's deployment stage.

Model Repair-based Defense. Model repair-based defense involves modifying or retraining the user's model to eliminate potential backdoors. Some methods prune or fine-tune a trained model to eliminate the backdoor neurons or disrupt the backdoor connections.Inspired by the observation that neurons related to backdoors are usually dormant when predicting benign samples, Liu et al. [19] proposed pruning these neurons to remove the hidden backdoors. More recently, Mu et al. [5] observed that for backdoored models, adversarial samples and their trigger samples have similar feature representations after being input into the model. Therefore, they fine-tuned the infected model using adversarial samples with correct labels. Some others aim to recover the probable triggers and determine anomalous triggers, and then put the determined triggers on benign samples to retrain the model, Wang et al. [20] reversed potential triggers for each class and then used anomaly detection to identify the trigger employed by the attacker and its target label. Wang et al. [21] formally defined and analyzed the triggers injected in different spaces and the inversion problem, then they proposed a unified framework to invert backdoor triggers based on the formalization of triggers. While model repair-based defense methods have shown promise, they still encounter limitation that need the modifications to the user's model.

Input Preprocessing-based Defense. This kind of methods preprocesses input samples before they are fed into the model, aiming to disrupt triggers in backdoored samples, ensuring they no longer match the hidden backdoor. Liu et al. [22] proposed the first input preprocessing defense, training an autoencoder with benign samples to reconstruct input samples and remove hidden triggers. However, this approach struggles with triggers weakly associated with the samples without prior knowledge, as the autoencoder cannot effectively distinguish triggers from the original samples. Doan et al. [23] proposed that use the Grad-CAM [24] to identify the trigger regions and remove them, followed by GAN-based restoration. However, this method often fails when attacks disperse triggers throughout the image. Some methods employed general data transformation methods (blurring[25], shrinking[26], etc.) to weaken the triggers. However, advanced attacks also utilize corresponding data augmentation strategies during the training process, and the triggers they use are more covert and robust. Recently, Sun et al. [27] proposed that detect possible trigger regions in the token space, then mask the trigger regions and use the masked autoencoder to restore the image. But when the trigger pattern is not located in a local region, it is difficult for their method to identify an appropriate mask. Shi et al. [6] proposed a Zero-shot Image Purification defense, applying linear transformations on the poisoned samples to destroy the triggers, and using pre-trained diffusion model to recover the missing information. However, linear transformations may be insufficient against invisible and robust triggers, potentially allowing their recovery. It is evident that existing defense methods have their respective limitations when confronted with more invisible and robust triggers. In contrast, our approach leverages the powerful capability of adversarial perturbation, effectively mitigating various invisible and robust triggers. Moreover, combining with blurring techniques further enhances the effectiveness of our approach, making our method adaptable against various backdoor attacks.

3.3. Adversarial attack

Adversarial attack are a form of threat during the interference phase of deep learning models. Attackers typically construct adversarial samples by introducing imperceptible perturbations to benign samples, intending to mislead the model into making incorrect predictions on these adversarial samples. Adversarial attacks can be categorized into non-targeted attacks and targeted attacks. The former aims to induce misclassification in the model without specifying a particular target, while the latter aims to lead the model to predict a specific target for the modified input. Goodfellow et al. [28] first proposed a algorithm called the Fast Gradient Sign Method (FGSM) for rapidly generating adversarial examples, which calculates perturbation to input data in a single step using the gradient of the loss function with respect to the input data with the

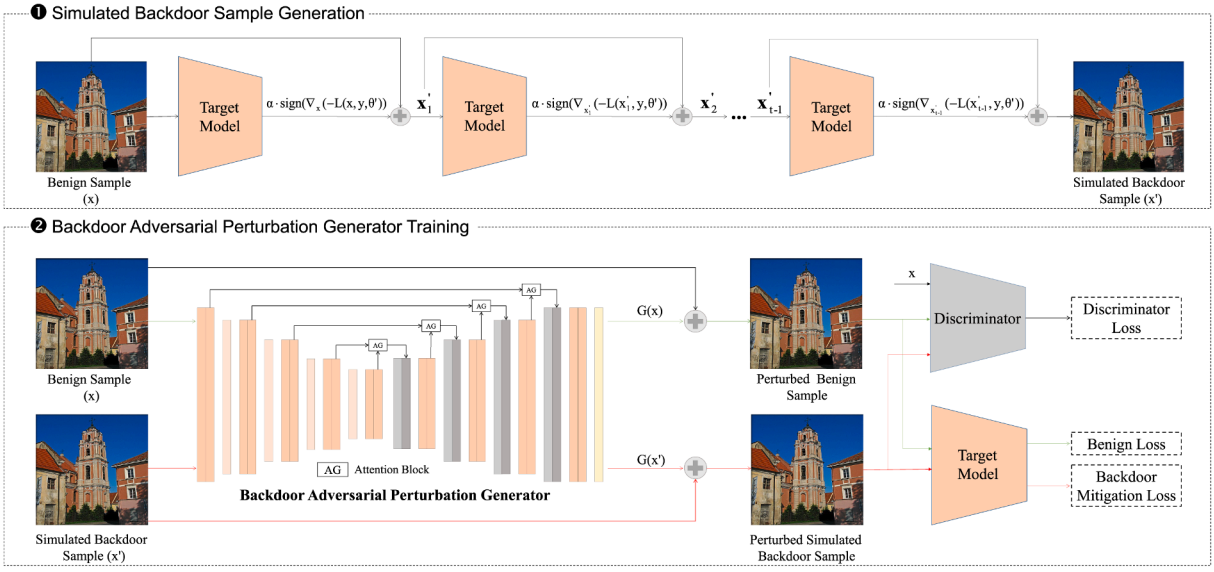


Fig. 2. The overview of the proposed Backdoor Adversarial Mitigation (BAM) involves two steps: Simulated Backdoor Sample Generation and Backdoor Adversarial Perturbation Generator Training. In the first step, we utilize an untargeted adversarial attack algorithm to generate simulated backdoor sample. In the second step, the Backdoor Adversarial Perturbation Generator is trained to generate perturbations that can mitigating the effect of backdoor triggers and rectifying the classification of the target model.

objective to maximize the loss. After that, Madry et al. [29] proposed an another typical adversarial attack algorithm which improves FGSM by generating an adversarial example iteratively. Wang et al. [30] proposed a Reflected Light Attack (RFLA), which generates adversarial examples by placing a color transparent plastic sheet and a paper cut aligned in front of a mirror to generate multiple colored geometries on the target object. Recently, Bai et al. [31] proposed a Neural Process-based black-box adversarial attack (NP-Attack), which leverages image structural information modeled by a Neural Process to explore the distribution of adversarial examples around benign inputs, thereby significantly reducing the number of queries required under the black-box setting. In contrast, backdoor attacks involve injecting triggers to force the model to classify backdoored samples into incorrect categories. In this paper, we propose to leverage adversarial perturbations to mislead backdoored models, thereby correcting the erroneous classifications of backdoored samples and enabling accurate predictions from the backdoored model.

4. Proposed method

To overcome the limitations of existing defense methods, we propose a novel backdoor mitigation approach that use the adversarial perturbation to weaken the triggers. We first introduce the threat model in our work and then present our Backdoor Adversarial Mitigation method.

4.1. Threat model

Our threat model is consistent with work [5]. The attacker has the capability to access training data or the training process, while the defender has access to the infected model and some additional clean datasets.

Attacker's Goal: Let $D_{benign} = \{x_i, y_i\}_{i=1}^{k}$ represent a unmodified benign training set and $C(x, \theta)$ is the benign DNN with parameter θ. The backdoor adversaries will design a trigger injection module $x^p = T(x, \Delta)$ to add carefully crafted triggers onto benign samples, then replace some benign samples in the D_{benign} with backdoored samples and change the labels of these replaced samples to the target label of the attack, which transform D_{benign} into a poisoned dataset $D_{backdoor} = \{x_i, y_i\}_{i=1}^{k-n} \cup \{x_i^p, l\}_{i=k-n+1}^{k}$, where l is the target label of attack, n represents the number of backdoored samples. The ratio of n to k is the poisoning ratio. Using $D_{backdoor}$ for training will result in an infected model $C(x, \theta')$. Giving an input x or x^p, the infected model satisfies the following conditions:

$$\begin{cases} C(x, \theta') = y \\ C(x^p, \theta') = l, \end{cases} \tag{5}$$

where y represents the ground truth label of x.

Defender's Goal: We assume that the defender has a small clean validation dataset and access to the user model $C(x, \theta')$. The defender is unaware of any potential trigger patterns or the backdoor in the model, and have no access to poisoned data or knowledge of the poisoning process. The defender's goal is introduce a preprocessing module to cleanse potential triggers within the input x, ensuring that the backdoor and triggers no longer match. It can be described as follows:

$$\begin{cases} C(P(x), \theta') = y \\ C(P(x^p), \theta') = y, \end{cases} \tag{6}$$

where $P(\cdot)$ is the preprocessing module, y represents the ground truth label of x.

4.2. Backdoor adversarial mitigation

Backdoor models exhibit abnormal responses to samples containing triggers, leading to specific outputs. Considering the capacity of adversarial perturbations to control model responses, we posit that they can be leveraged to rectify the anomalous behavior in backdoor models.

In this paper, we propose a novel backdoor defense scheme named Backdoor Adversarial Mitigation (BAM). As depicted in Fig. 2, BAM comprises two steps: Simulated Backdoor Sample Generation and Backdoor Adversarial Perturbation Generator Training. In the first step, we utilize untargeted adversarial attacks to generate simulated backdoor sample that exhibit similar features to the backdoored sample. In the second step, we develop an Backdoor Adversarial Perturbation Generator to produce mitigation perturbations, ensuring that the generated perturbations weaken the trigger effects while not impacting the classification of benign samples. The optimization of the Backdoor Adversarial Perturbation Generator is based on Backdoor Mitigation loss, Benign Loss, Discriminator Loss, and the Perturbation Loss. During the deployment phase, we further introduce a Gaussian Blur operation to enhance our BAM on visible triggers. Detailed descriptions are provided below.

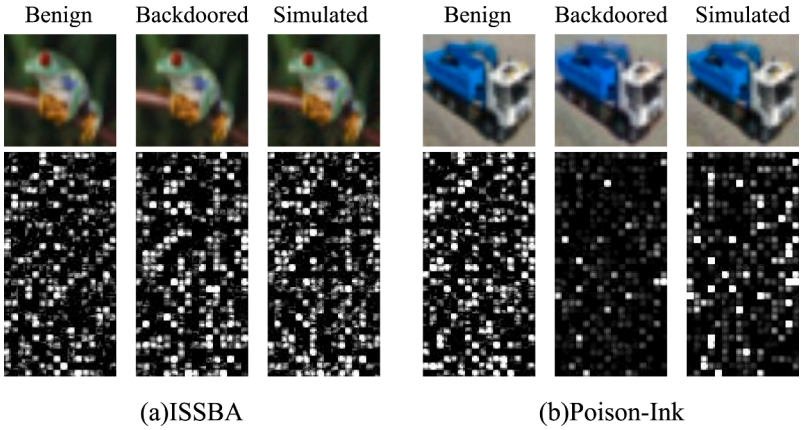


Fig. 3. The features maps for Benign sample, backdoored sample and simulated backdoor sample under ISSBA [13] attack and Poison-Ink [14] attack.

Simulated Backdoor Sample Generation. This step is designed to generating simulated backdoor sample. Our method aims to leverage adversarial perturbations to mitigate the triggers hidden in the backdoored samples. However, the unavailability of actual backdoored samples poses a challenge in optimizing adversarial perturbations. To address this issue, following work[5], we utilize adversarial examples as simulated backdoor samples. As illustrated in Fig. 3, we extracted feature maps from the final convolutional layer output of models attacked by ISSBA [13] and Poison-Ink [14]. The feature maps of simulated backdoor samples closely resemble those of the backdoored samples. Consequently, the generated simulated backdoor samples can effectively approximate backdoored samples and be utilized to train our Backdoor Adversarial Perturbation Generator.

Similar to the approach described in work[5], we conduct untargeted adversarial attacks on the target model $C(x,\theta')$ to generate simulated backdoor sample x' from benign sample x, which employs the PGD (Projected Gradient Descent) adversarial attack algorithm. PGD is an iterative and strong attack method that maximizes the model's prediction error within a limited perturbation range and is widely regarded as a classic and robust technique for generating effective adversarial examples. We first set the initial simulated backdoor sample x'_0 to benign sample x. Then, we iterate t times to generate perturbation such that the simulated backdoor sample $x' = x'_t$ could maximizes the classification loss, as illustrated below:

$$\delta_t = \mu \cdot sign(\nabla_{x'_t}(-L_{CE}(C(x'_t,\theta'),y))), \tag{7}$$

$$x'_{t+1} = clip_\varepsilon(x'_t + \delta_t), \tag{8}$$

where L_{CE} represents the cross-entropy loss, y is the label of the benign sample x, μ is the update step, the $sign()$ represents the sign function, ∇ means gradient calculation, and ε is the valid range of pixel values. In our experiments, we set the μ to $6/(255\times 5)$ and the times of iteration t to 7. In Fig. 2, We represent the $L_{CE}(C(x'_t,\theta'),y)$ as $L(x'_t,y,\theta')$.

Backdoor Adversarial Perturbation Generator Training. This step involves training an Backdoor Adversarial Perturbation Generator to mitigate the effects of triggers. We design the generator based on the U-Net architecture [32]. Specifically, the Backdoor Adversarial Perturbation Generator comprises an encoder and a decoder. The encoder extracts features from the samples, while the decoder further utilizes these features to generate subtle perturbations that interfere with the output of the target model. Additionally, shallow features from the encoder layer are passed to the decoder to aid in perturbation generation. To enhance the network's sensitivity to triggers, we introduce an Attention module to selectively filter shallow features, thereby preserving those relevant to the trigger. The computational process of the model is outlined as follows.

Firstly, the benign sample x and simulated backdoor sample x' are input into the Backdoor Adversarial Perturbation Generator. The generator G produces corresponding perturbations, $G(x)$ and $G(x')$, for the two samples. Subsequently, the perturbed samples $x+G(x)$ and $x'+G(x')$ are fed into the target model. The perturbation $G(x)$ should not affect the normal classification of x in the target model, while the perturbation $G(x')$ should correct the classification of x' in the target model. To ensure the effectiveness of the generated perturbations, we use the following four loss functions to optimize the Backdoor Adversarial Perturbation Generator.

Benign Loss: For benign sample, the perturbation generated by the Backdoor Adversarial Perturbation Generator should not affect their classification. Therefore, we input the perturbed samples into the target model and compute the cross-entropy loss as follows:

$$L_{benign} = L_{CE}\big(C\big(x+G(x),\theta'\big),y\big), \tag{9}$$

where L_{CE} represents the cross-entropy loss, y is the label of the benign sample x.

Backdoor Mitigation Loss: For simulated backdoor sample, the perturbation generated by the Backdoor Adversarial Perturbation Generator should mitigate its backdoor features, enabling them to be classified normally. The loss function is as follows.

$$L_{bm} = L_{CE}\big(C\big(x'+G\big(x'\big),\theta'\big),y\big). \tag{10}$$

Discriminator Loss: To preserve the visual quality of perturbed samples, we introduce a discriminator (D) to train alongside the Backdoor Adversarial Perturbation Generator. The optimization objective of the discriminator is to classify perturbed samples and benign sample, while the generator aims to generate perturbed samples that are indistinguishable. The discriminator is optimized using binary cross-entropy loss, and the loss function for the Backdoor Adversarial Perturbation Generator is as follows:

$$\begin{aligned} L_{dis} = {} & \mathbb{E}_x log(D(x)) + \mathbb{E}_x log(1-D(x+G(x))) \\ & + \mathbb{E}_{x'} log\big(1-D\big(x'+G\big(x'\big)\big)\big). \end{aligned} \tag{11}$$

Perturbation Loss: To constrain the magnitude of the perturbation and stabilize the training of the generator, we utilize a soft hinge loss on the L_2 norm, defined as:

$$L_{pert} = max\big(0,||G(x)||_2 - c\big) + max\big(0,||G\big(x'\big)||_2 - c\big), \tag{12}$$

where c denotes the perturbation threshold.

Finally, we combine all the aforementioned losses to train the Backdoor Adversarial Perturbation Generator:

$$L = L_{benign} + \alpha L_{bm} + \beta L_{dis} + \lambda L_{pert}, \tag{13}$$

where α, β and λ are loss weights used for balancing designed losses.

4.3. Combine with blur

The Blur operation is incorporated to enhance BAM's capability in eliminating visible triggers. During the adversarial training process, we

constrain the magnitude of perturbations to improve the visual quality of perturbed samples. However, for visible triggers, subtle perturbations are ineffective in disrupting them. To enhance the impact on visible triggers, following [25],we introduce the Gaussian Blur [33], it has minimal impact on the classification of benign samples compared to other data transform methods, and it can effectively weaken visible triggers. In contrast, methods like cropping or compression, although capable of reducing trigger visibility to some extent, tend to severely degrade image quality and semantic content, leading to a substantial drop in clean classification accuracy. Specifically, During the deployment phase, we first apply Gaussian Blur to the image to weaken the visible triggers while minimally affecting normal classification. Subsequently, we use the Backdoor Adversarial Perturbation Generator to generate perturbations for the blurred sample. This approach enables us to further integrate these perturbations into the samples, effectively weakening both visible and invisible triggers, thereby achieving robust backdoor defense.

5. Experiments

In this section, we first introduce our experimental settings, and then compare our method with existing state-of-the-art defense methods. After that, we conduct ablation studies to prove the effectiveness of designed components.

5.1. Experimental setup

Datasets. We evaluate all relevant backdoor and defense methods on three datasets: CIFAR-10 [34], GTSRB [35], and Imagenette [36]. CIFAR-10 comprises $50,000$ colored training images and $10,000$ testing images sized 32×32, and it is categorized into 10 classes. GTSRB consists of $39,209$ colored training images and $12,630$ testing images covering 43 different traffic signs. With images of varying sizes, we resize all of them to 32×32. Imagenette, a subset of the ImageNet dataset, it offers 10 classes with $13,000$ images of varying sizes, which we resize to 224×224, and images are split to training and validation sets following predefined split ratio. These datasets cover a broad spectrum of tasks and complexities, highlighting the wide applicability and effectiveness of our approach. Similar to prior work [5], we assume access to the test sets of these datasets. We utilize $70\,\%$ of the test set for training and the remaining $30\,\%$ for verifying. The training data is augmented with random crop, random rotation, and Blur. During verifying, we exclude samples belonging to the target class, as attackers typically place triggers on samples outside the target class.

Backdoor Attack Settings. We evaluate the proposed method on six well-known backdoor attacks:

- **BadNets** [8]: It overlays a visible local pixel block as trigger on the lower right corner of benign samples. We use a 3×3 grid as the trigger on CIFAR-10 and GTSRB datasets, and a 6×6 grid as the trigger on the Imagenette dataset.
- **Blended** [11]: It blends a pattern with certain transparency on samples as trigger. We use a hello-kitty image as the trigger and blend it with benign samples with a ratio of 0.1.
- **Input-aware** [10]: It trains a trigger generator to produce triggers for various samples, but the triggers are visible.
- **WaNet** [12]: It uses small and smooth distortion fields to distort samples in a visually indistinguishable way.
- **ISSBA** [13]: It encodes an invisible string specified by the attacker into the sample using an encoder-decoder network, meeting the criteria of invisibility and sample specificity.
- **Poison-Ink** [14]: It extract the edge structure of benign samples and fills this area with poisonous ink information to generate the triggers, then embedded triggers into the benign samples using a deep injection network. These triggers are also invisible and sample-specific, but more robust.

For the attacks mentioned, we employ BackdoorBench [37] to ensure fairness in training for comparison purposes. However, since BackdoorBench does not provide a training method for Poison-Ink, we utilize the default settings provided in the source code by the author. We train the corresponding target model for all the aforementioned attacks on PreAct-ResNet18 [38], with a poisoning ratio of $10\,\%$. Additionally, all training data are augmented with random crop and rotation to enhance the robustness of the trigger.

Compared Methods. We compare with six state-of-the-art backdoor defense methods:

- **IP** [22]: This method removes triggers by training autoencoder to reconstruct samples. We use autoencoder described in [39], and train with the same training data as our method.
- **ShrinkPad** [26]: ShrinkPad first shrinks samples and then randomly pads them to the original size. We adjusted the authors' implementation to our framework, using fill size of 8 for CIFAR-10 and GTSRB, and 16 for Imagenette.
- **Blur** [25]: Blur uses Gaussian Blur to weaken triggers. We implemented it with 3×3 Gaussian kernel of standard deviation 0.5 for CIFAR-10 and 1.0 for GTSRB and Imagenette.
- **Februus** [23]: Februus uses GradCAM [24] to locate triggers and restores missing parts with GAN models. We selected the appropriate activation layer and threshold for each attack, and repaired the removed areas using the GAN models provided by the author. For the Imagenette dataset, since there is no inpainting GAN in the authors' code, we used the "inpaint" function from OpenCV instead.
- **BDMAE** [27]: BDMAE identifies the trigger regions in the input image and masks them, then uses a masked autoencoder to restore the image. We apply the BDMAE by using its official code, and use the base autoencoder provided by the author for repairing the masked images.
- **ZIP** [6]: ZIP applies linear transformations on the poisoned image to destroy the triggers, and then uses a pre-trained diffusion model to recover the missing semantic information. As provided by the authors, for CIFAR-10, we applied both blur and gray-scale conversion as linear transformations, while for GTSRB and Imagenette, solely applied blur. The kernel size of average pooling is set to 2.0, and the hyperparameter λ is set to 10.

Implementation Details. For the target model trained for different backdoor attacks on three datasets, the adversarial samples are generated with those models and regarded as the simulated backdoor samples to train the Backdoor Adversarial Perturbation Generator in our BAM method. Generator contains five convolutional blocks with 3×3 convolutions and ReLU activations, with output channels of 64, 128, 256, 512, and 1024, each followed by 2×2 max-pooling (except the last). The decoder mirrors this structure with upsampling, attention modules, and concatenation from encoder layers, followed by two 3×3 convolutions at each stage, gradually reducing the output channels to 512, 256, 128, and 64. The output layer uses a 1×1 convolution to produce a 3-channel image, followed by a Sigmoid or Tanh activation. The discriminator consists of a series of Conv2d + ReLU + MaxPool layers, with progressively increasing channels (64→2048), followed by an adaptive average pooling to 1×1, and three fully connected layers ending in a Sigmoid function to output the discrimination probability. Using a classification network consisting of six convolutional layers and three fully connected layers as the discriminator. During the training of the generator and discriminator, we utilize the Adam optimizer with an initial learning rate of 0.001. Additionally, we employ ReduceLROnPlateau to automatically adjust the learning rates, using the classification accuracy of perturbed simulated backdoor samples and loss of discriminator as metric of generator and discriminator respectively. The number of training epochs is fixed at 100. Furthermore, we set the loss hyperparameters α, β, and λ to 1, 1, and 1, respectively. The perturbation thresholds c is set to 0.4 for CIFAR-10, 1.0 for GTSRB and Imagenette, this is because on CIFAR-10 a relatively small c was sufficient to preserve clean accuracy, whereas

slightly larger values were required for GTSRB and Imagenette due to their more complex data distributions.

Evaluation Metrics. We assess the efficacy of defense mechanisms using three metrics:

- **Attack Success Rate (ASR)**: The proportion of backdoored examples misclassified as the target label.
- **Clean Accuracy (CA)**: The accuracy of the model on perturbed benign samples.
- **Backdoor Accuracy (BA)**: The accuracy of the model on perturbed backdoored samples.

5.2. Defense performance

This subsection evaluates the defense effectiveness of our proposed algorithm. The Clean Accuracy (CA), Attack Success Rate (ASR), and Backdoor Accuracy (BA) metrics are measured. We compare our results with state-of-the-art methods on three datasets.

CIFAR-10. Table 1 presents our defense performance on the CIFAR-10 dataset. Across all attacks, our BAM significantly reduce their ASR to an average of 3.56 %, and increases their BA by over 81 % on average, outperforming other algorithms overall. Although IP performs well in dealing with some invisible attacks, it cannot handle attacks based on hybrid strategies, such as Blended. For Februus, it is more effective against attacks like BadNets and ISSBA, but performs poorly against other attacks because their triggers can still function effectively even when partially absent. Shinkpad demonstrates well performance against BadNets and WaNet, but showing weaker efficacy against attacks that are insensitive to changes in shape and position, such as Poison-Ink. Blur is effective against attacks that are sensitive to changes in pixel values, such as BadNets and InputAware, but its effectiveness is diminished against invisible attacks because the triggers of these attacks are more robust and tolerant to changes in pixels and shapes. BDMAE has a high CA and performs well against BadNets attacks, but its effectiveness is limited when dealing with other attacks where the trigger is not localized. ZIP defense method is effective against visible attacks like BadNets and Input-aware, it also performs well in dealing with ISSBA and Poison-Ink, but it proves ineffective against Blended and WaNet. Our BAM has demonstrated excellent performance in countering various invisible attacks. Although its effectiveness is limited against more visible attacks, the combination with Blur has enhanced the performance of our method against both visible attacks and invisible attacks.

GTSRB. Table 2 presents our defense performance on the GTSRB dataset. Compared to CIFAR-10, the image features of GTSRB are simpler and easier to learn. However, this also provides the model with more capacity to learn trigger features, thereby making some attacks more challenging to defend against. Despite this, our defense consistently reduces the average ASR to 1.57 % across all attacks. Furthermore, the average CA remains stable or even improves, while the average BA increases by over 94 %. IP still demonstrates strong performance against WaNet and ISSBA, but its effectiveness against Poison-Ink and Input-aware has declined. Februus continues to exhibit good performance against ISSBA, but its effectiveness against BadNets has waned. Shinkpad also shows a decline performance on WaNet. Similarly, ZIP's effectiveness decreases against Poison-Ink, yet it showcases robust defensive performance against WaNet. In contrast, our approach maintains robust performance against all attacks.

Imagenette. Table 3 presents the results of our approach on Imagenette dataset. Due to the larger image size, the trigger size also increases correspondingly, making some attacks more challenging to defend against. For all attacks, our defense still reduces the average ASR to 4.35 %, with CA dropping by less than 6 % on average, the average BA increases by over 72 %. IP performs well to WaNet and Input-aware, but effectiveness to ISSBA and Poison-Ink declined. The effectiveness of Februus for ISSBA also has decreased. ShrinkPad shows a decrease to Input-aware and WaNet. BDMAE still maintains good CA performance, but it also

Table 1
Comparison with the state-of-the-art defense methods on CIFAR-10 dataset(%).

Attacks	No defense			IP [22]			Februus [23]			ShrinkPad [26]			Blur [25]			BDMAE [27]			ZIP [6]			BAM(ours)			Blur+BAM(ours)		
	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑
BadNets [8]	91.46	99.96	0.04	82.74	6.84	77.86	78.53	12.86	78.16	81.12	12.16	79.16	88.91	9.39	82.30	**89.39**	**0.96**	**89.25**	78.06	4.24	77.43	88.91	100.00	0.00	86.17	7.45	81.60
Blended [11]	93.39	99.63	0.30	78.34	98.82	1.00	89.46	91.72	1.18	87.47	46.20	44.46	91.11	97.05	2.66	**92.03**	99.41	0.54	81.36	99.62	0.37	88.93	42.58	51.59	87.12	**4.43**	**80.59**
Input-aware [10]	90.72	94.05	5.80	79.56	39.95	36.40	87.73	84.04	14.27	88.84	58.10	37.44	87.25	**3.10**	82.04	**89.32**	81.92	16.85	79.85	15.85	65.89	88.32	4.91	**84.22**	85.84	3.54	84.11
WaNet [12]	89.87	98.45	1.44	85.99	2.77	80.45	88.40	78.38	18.70	82.85	11.71	76.02	66.85	99.92	0.07	**88.96**	97.82	2.14	75.00	96.88	1.56	88.54	**1.96**	**86.36**	86.58	2.25	81.97
ISSBA [13]	93.13	100.00	0.00	82.74	**0.22**	82.71	88.10	18.44	62.12	87.62	31.86	61.31	90.84	98.30	1.55	**92.14**	100.00	0.00	81.59	2.88	80.19	90.20	1.15	**90.10**	88.10	0.96	88.32
Poison-Ink [14]	91.54	99.89	0.11	85.63	3.32	81.69	88.59	97.36	2.30	84.48	60.08	34.94	90.12	60.23	36.29	90.22	99.72	0.19	80.14	2.76	75.90	**90.42**	7.18	80.55	87.88	**2.32**	**82.90**
Avg	91.69	98.66	1.28	82.50	25.32	60.02	86.80	63.80	29.46	85.40	36.69	55.56	85.85	61.33	34.15	**90.34**	79.97	18.16	79.33	37.04	50.22	89.22	26.30	65.47	86.95	**3.56**	**83.25**

Table 2
Comparison with the state-of-the-art defense methods on GTSRB dataset(%).

Attacks	No defense			IP [22]			Februus [23]			ShrinkPad [26]			Blur [25]			BDMAE [27]			ZIP [6]			BAM(ours)			Blur + BAM(ours)		
	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑
BadNets [8]	97.40	100.00	0.00	94.81	**0.00**	91.54	88.63	84.63	13.65	86.72	10.15	83.51	96.21	**0.00**	96.10	95.83	0.16	95.84	85.26	0.53	83.28	98.67	100.00	0.00	**99.23**	**0.00**	**99.25**
Blended [11]	98.51	99.95	0.05	94.70	98.41	1.54	97.14	99.79	0.21	93.80	82.81	16.03	97.37	79.58	19.10	98.49	99.92	0.08	91.83	99.92	0.07	97.96	50.94	47.92	**99.25**	**4.85**	**88.65**
Input-aware [10]	98.28	97.93	2.07	96.60	63.10	22.68	93.40	73.39	26.24	93.59	41.32	76.70	97.13	**0.00**	75.48	98.01	96.98	2.65	84.53	71.21	20.89	**98.78**	28.62	70.90	98.35	**0.00**	**97.85**
WaNet [12]	97.30	99.81	0.19	97.40	6.84	90.43	97.09	99.71	0.29	76.91	84.71	14.97	65.60	99.81	0.19	96.80	99.62	0.38	85.50	2.47	85.02	**98.86**	0.27	98.57	98.28	**0.00**	**97.80**
ISSBA [13]	98.25	99.76	0.24	96.52	3.63	93.64	87.49	16.72	72.57	87.25	92.66	7.02	96.98	99.54	0.46	97.38	90.00	9.89	86.16	0.05	85.95	**98.33**	0.21	**96.95**	97.27	**0.00**	96.29
Poison-Ink [14]	98.83	100.00	0.00	97. 61	44.39	54.18	97.58	96.68	0.05	94.39	99.18	0.82	98.48	100.00	0.00	98.70	99.89	0.11	85.75	48.38	50.47	**98.70**	5.07	93.56	98.20	**4.56**	**95.41**
Avg	98.10	99.58	0.43	96.27	36.06	59.00	93.56	78.49	18.84	88.78	68.47	33.18	91.96	63.16	31.89	97.54	81.10	18.16	86.51	37.09	54.28	**98.55**	30.01	67.98	98.43	**1.57**	**95.88**

Table 3
Comparison with the state-of-the-art defense methods on Imagenette dataset(%).

Attacks	No defense			IP [22]			Februus [23]			ShrinkPad [26]			Blur [25]			BDMAE [27]			ZIP [6]			BAM(ours)			Blur + BAM(ours)		
	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑
BadNets [8]	89.30	98.03	1.97	82.72	65.54	32.96	83.66	46.95	49.01	87.89	36.06	60.28	82.35	2.72	81.78	**88.26**	1.13	**87.84**	85.72	**0.85**	83.84	86.47	98.56	1.03	83.94	2.35	82.81
Blended [11]	88.64	98.69	1.31	78.78	96.24	3.29	82.44	93.24	1.97	86.76	96.06	3.85	80.09	78.50	18.22	**87.88**	98.57	1.41	84.69	80.46	18.03	85.82	63.19	31.74	82.53	**8.73**	**68.54**
Input-aware [10]	83.28	96.53	3.38	75.49	4.69	49.77	81.60	72.38	26.20	81.41	96.15	3.76	63.10	2.07	57.37	82.07	95.40	4.41	75.11	7.23	67.69	**82.53**	**0.84**	**81.31**	79.72	1.78	74.27
WaNet [12]	86.10	98.50	1.41	80.09	6.48	74.37	83.38	92.76	2.44	66.29	99.53	0.38	53.08	100.00	0.00	**85.25**	97.56	2.35	63.76	93.05	6.10	76.15	8.08	76.71	77.74	**3.00**	**75.87**
ISSBA [13]	89.20	98.12	1.60	73.80	39.25	45.63	83.66	55.21	40.00	86.57	97.09	2.44	80.68	93.24	6.01	**87.41**	97.75	1.97	77.37	59.91	35.96	84.04	52.02	41.03	83.47	**1.97**	**81.22**
Poison-Ink [14]	80.78	85.54	11.94	79.76	53.05	43.69	70.42	80.37	16.62	73.79	83.46	14.57	76.30	85.54	11.94	**79.81**	85.07	12.30	79.06	31.55	59.43	76.71	9.23	70.32	76.90	**8.27**	**71.26**
Avg	86.22	95.90	3.60	78.44	44.21	41.62	80.86	73.49	22.71	80.45	84.73	14.21	72.60	60.35	29.22	**85.11**	79.25	18.38	77.62	45.51	45.18	81.95	38.65	50.36	80.72	**4.35**	**75.66**



Fig. 4. The Grad-CAM comparison with the defense methods existing state-of-the-art defense methods on imagenette dataset. The first row shows the Grad-CAM of benign samples, the second row shows the Grad-CAM of backdoored samples, and the remaining rows show the Grad-CAM of backdoored samples after defense.

remains ineffective against most attacks. The effect of ZIP to visible attacks has improved, but its' effect of invisible attacks declined. Compared to baseline defenses, our method still shows smaller CA degradation and larger ASR reduction for all attacks, while maintaining a higher BA.

5.3. Defense visualization

In this section, we demonstrate the efficacy of different defense methods using Grad-CAM [24]. We feed the backdoored samples that have been defended into the target model and then use features extracted from the last fully connected layer to calculate the Grad-CAM. As shown in Fig. 4, the Grad-CAM of various backdoored samples after being defended by our method is most similar to those of the original benign samples. This significant resemblance underscores the effectiveness of our method in weakening the trigger features of backdoored samples. Other defense methods are either unable to weaken the trigger or they weaken the trigger while greatly affecting areas other than the trigger, especially for invisble and robust backdoor attack methods (WaNet, ISSBA, and Poison-Ink). In contrast, our method has minimal impact on the backdoored samples except in the trigger regions.

5.4. Ablation study

In this section, we perform ablation studies on the CIFAR-10 dataset to evaluate the effectiveness of the designed components.

Effectiveness Of Each Loss Term. The Discriminator Loss and Perturbation Loss are designed to stabilize GAN training and improve image quality. To prove their effectiveness, we conduct an ablation study to evaluate the contribution of them on CIFAR-10 dataset. The result is shown in Table 4. While removing each component leads to a

Table 4
Defense performance under different components of losses on CIFAR-10 dataset (%).

Attacks→		WaNet			ISSBA			Poison-Ink		
L_{dis}	L_{pert}	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑
		85.92	1.11	82.34	87.21	1.77	86.88	72.01	1.76	63.77
✓		85.77	1.74	82.67	85.77	1.00	86.43	86.60	1.23	82.60
	✓	86.58	0.89	82.08	87.21	1.11	86.22	69.21	1.38	59.18
✓	✓	86.58	2.25	81.97	88.10	0.96	88.32	87.88	2.32	82.90

Table 5
Performances of our method against backdoor attacks under different model architectures (%).

Datasets		No defense			Ours		
	Attacks	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑
CIFAR-10	CL	90.82	99.83	0.17	86.42	7.08	84.94
	ISSBA	89.82	100.00	0.00	84.94	0.77	84.35
	Poison-Ink	88.02	100.00	0.00	85.60	2.76	82.06
GTSRB	CL	98.28	95.26	4.59	97.32	1.90	92.79
	ISSBA	98.54	99.66	0.21	97.88	0.77	95.97
	Poison-Ink	98.17	100.00	0.00	97.53	2.25	95.01

Table 6
Ablation of the attention block (%).

Attacks	No defense			w/o Attention			w/ Attention		
	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑
WaNet	89.87	98.45	1.44	67.40	4.50	62.93	86.58	2.25	81.97
ISSBA	93.13	100.00	0.00	65.45	8.54	49.56	88.10	0.96	88.32
Poison-Ink	91.54	99.89	0.11	66.09	8.63	61.37	87.88	2.32	82.90

Table 7
Defense performance under different proportions of training data on CIFAR-10 dataset (%).

Attacks	Training data ratio	1.0 (Default)			0.5			0.1		
	Defense	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑
WaNet	IP	85.99	2.77	80.45	85.00	2.25	80.23	71.18	1.51	69.07
	Blur + BAM(ours)	86.58	2.25	81.97	86.14	2.70	80.64	85.99	2.48	79.97
ISSBA	IP	82.74	0.22	82.71	81.74	0.44	81.34	59.68	0.18	58.50
	Blur + BAM(ours)	88.10	0.96	88.32	86.73	1.74	86.44	87.58	1.03	87.29
Poison-Ink	IP	85.63	3.32	81.69	84.41	2.32	80.19	65.92	5.17	63.76
	Blur + BAM(ours)	87.88	2.32	82.90	89.03	4.38	81.89	87.46	3.59	80.62

significant drop in defense in certain cases, this underscores the importance of each loss component in effectively mitigating different types of attacks.

Defense Effectiveness Under Different Model Architectures. To evaluate the performance of our defense method across different model architectures, we conducted additional experiments on the CIFAR-10 and GTSRB datasets using alternative network structures. Specifically, we adopted the VGG-19 model for CIFAR-10 and the ResNet-50 model for GTSRB. We also included experiments against clean-label attack(CL)[40]. As shown in the Table 5, the results demonstrate that our method consistently achieves strong defense performance across different architectures and effectively mitigates clean-label attacks as well. These findings highlight the strong generalization ability and broad applicability of our defense approach.

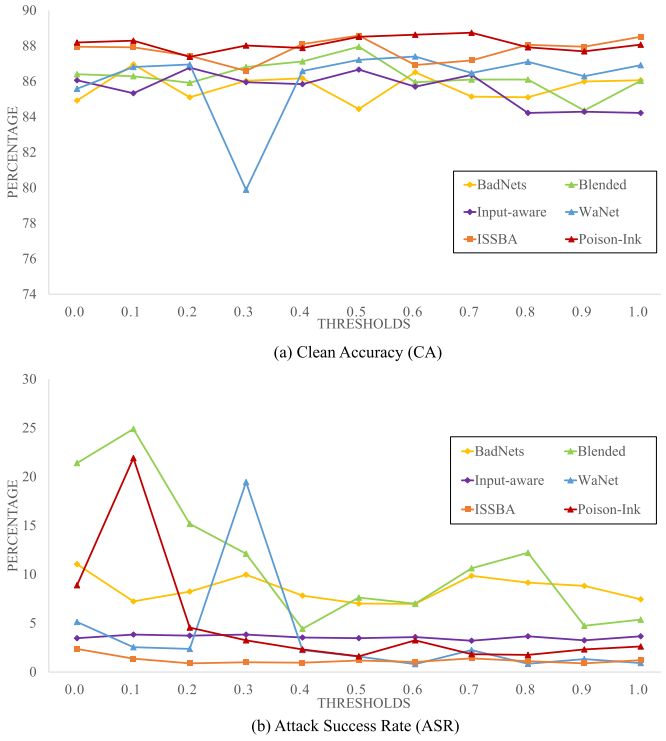
Effectiveness Of Attention Block. To enhance the network's sensitivity to triggers, we introduce an Attention module to the Backdoor Adversarial Perturbation Generator. To demonstrate the effectiveness of the attention module, we conducted ablation experiments on the CIFAR-10 dataset. In these experiments, we used a standard U-net structure as the Backdoor Adversarial Perturbation Generator for the model without the attention module. As shown in the Table 6, incorporating the attention module significantly improved the classification accuracy on clean samples and effectively reduced the attack success rate for certain attacks.

Defense Effectiveness Under Different Training Ratios. The training ratio is related to the robustness of the backdoor defense scheme.Table 7 shows the results of IP and our defense under different training ratios, a training ratio of 1.0 means the defender can use 70 % of the full test dataset (our default setting), while a ratio of 0.5 indicates that only 35 % is available for training, and so on. It can be observed that IP exhibits larger decreases in CA and BA with fewer training data. This is attributed to the inadequate training of the autoencoder on smaller data sets, resulting in insufficient reconstruction of input samples. In contrast, our defense demonstrates only minor decreases in CA and BA, accompanied by a slight increase in ASR as the training data decreases. This suggests the robustness of our defense against a limited amount of training data.

Defense Effectiveness Under Different Loss Function Hyperparameters. To investigate the impact of loss function hyperparameters on defense performance, we conducted experiments on our method using different loss function hyperparameters values on the CIFAR-10 dataset. Table 8 illustrates the defense performance. It can be observed that reducing the weight of the Backdoor Mitigation Loss, compared to

Table 8
Defense performance under different loss function hyperparameters on CIFAR-10 dataset (%).

Attacks→			WaNet			ISSBA		
λ_1	λ_2	λ_3	CA↑	ASR↓	BA↑	CA↑	ASR↓	BA↑
0.5	1	1	86.18	7.39	75.42	86.81	4.32	80.71
1	0.5	1	84.63	5.58	75.98	84.55	1.48	80.89
1	1	0.5	84.70	1.51	81.37	85.80	1.62	81.12
2	1	1	82.52	1.63	78.64	83.37	0.37	80.19
1	2	1	86.70	8.01	76.60	87.25	6.25	76.64
1	1	2	87.13	9.90	74.46	88.58	4.10	79.97
1	1	1	86.58	2.25	81.97	88.10	0.96	88.32



(a) Clean Accuracy (CA)

(b) Attack Success Rate (ASR)

Fig. 5. Defense performance under different different perturbation thresholds c on CIFAR-10 dataset.

decreasing the other two loss terms, results in a higher CA but simultaneously leads to a lower BA. This suggests that the generated perturbations tend to affect regions unrelated to the trigger, thereby exerting a weaker influence on the backdoor behavior. In contrast, reducing the Discriminator Loss or Perturbation Loss causes a more significant drop in both CA and BA, indicating that the perturbations exert greater influence on the classification of benign samples. On the other hand, increasing the weight of the Backdoor Mitigation Loss raises the ASR and BA, implying that the perturbation generator focuses more intensively on the trigger regions. An increase in the Discriminator Loss or Perturbation Loss leads to a reduction in BA and a concomitant increase in ASR, as it prevents the perturbation generator from concentrating on the trigger area. Overall, when the contributions of the three losses are balanced, the model achieves the highest CA and BA under both attack scenarios while maintaining a low ASR, suggesting that a well-balanced loss configuration enables the perturbation generator to optimally focus on the trigger region while minimizing the impact on benign sample classification.

Defense Effectiveness Under Different Perturbation Thresholds c**.** To investigate the impact of perturbation thresholds c on defense performance, we conducted experiments on our method using different c values on the CIFAR-10 dataset. Fig. 5 illustrates the defense performance. The results indicate that setting the perturbation threshold too small can result in higher ASR for certain attacks. This occurs because when the perturbation is too small, the triggering features in some samples are not adequately mitigated. As shown in Fig. 5-b, for attacks like Input-aware and ISSBA, once c surpasses a relatively small threshold, the defense effectiveness rapidly reaches a high level, and further increasing c does not significantly reduce the ASR. This is because although these attacks are relatively robust, our defense can effectively mitigate them with moderate perturbations. Notably, even when c is set to zero, the defense still demonstrates significant effectiveness. This is because the soft hinge loss only penalizes perturbations exceeding the threshold, and the perturbation magnitude is also naturally regulated by other training objectives, so the generator still produces small but non-zero perturbations. These perturbations are sufficient to disrupt backdoor triggers while maintaining minimal visual distortion. For other attack types, larger perturbations are needed to weaken the trigger's effect, so changes in c have a more noticeable impact. Additionally, the choice of the perturbation threshold has slight impact on the classification accuracy of benign samples. We have selected appropriate perturbation thresholds for different datasets to achieve the optimal defense performance.

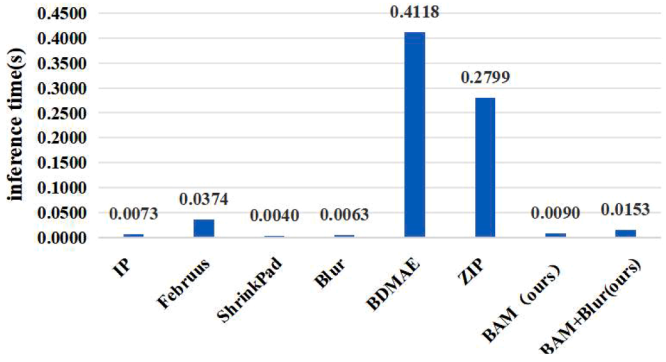


Fig. 6. Average inference time per image for different defense methods on CIFAR-10 dataset.

5.5. Evaluation on inference time

To illustrate deployment feasibility, the run-time of defense methods should be considered. We therefore evaluated various defenses on the CIFAR-10 dataset by processing 100 images with each method and measuring the inference time, defined as the average time required by the classifier to perform inference on a single image. As shown in Fig. 6, BAM requires 0.0090 s per image, while BAM + Blur takes 0.0153 s. Although slightly higher than lightweight methods such as IP and Blur, both are substantially faster than BDMAE (0.4118 s) and ZIP (0.2799 s). Given the significant robustness improvement, this additional latency is acceptable for practical deployment.

6. Conclusion

In this paper, we introduce Backdoor Adversarial Mitigation (BAM), a defense mechanism designed to combat robust and invisible triggers. BAM utilizes an untargeted adversarial attack algorithm to initially simulate backdoored samples. Following this, benign samples and simulated backdoor samples are used to train the Backdoor Adversarial Perturbation Generator, enabling the generated perturbations diminishing trigger functionality. The proposed BAM undergoes comprehensive evaluation across three datasets employing six distinct backdoor attack algorithms. Results illustrate that BAM exhibits significant superiority over existing defense methods in mitigating robust and invisible triggers. Although BAM demonstrates strong defensive performance, it still exhibits certain limitations. On one hand, the current approach relies

on access to a subset of clean data and model gradients to construct defense strategies. This white-box assumption poses challenges in fully zero-knowledge black-box settings. Future research could therefore focus on developing black-box defense methods. For instance, techniques such as Zeroth-Order Optimization or transfer-based attacks employing substitute models could enable the generation of adversarial perturbations without requiring direct access to the target model's gradients. Exploring these directions would enhance the generality of BAM and represents a promising avenue for future work. On the other hand, the current defense framework is not well adapted to the distributed training nature of federated learning. In sensitive domains such as medical image analysis and financial risk assessment, the conflict between data localization requirements and global model optimization hinders the practical deployment of defense mechanisms. Consequently, investigating backdoor defense strategies specifically designed for such distributed and privacy-preserving settings will also be a key focus of our future research.

Data availability

Data will be made available on request.

CRediT authorship contribution statement

Run Lu: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Peipeng Yu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Zhitao Huang:** Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Zhihua Xia:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Methodology, Funding acquisition; **Xiaoyan Jiang:** Validation, Resources, Project administration, Formal analysis, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China under grant numbers, U23B2023 and 62472199, Guangdong Key Laboratory of Data Security and Privacy Preserving under Grant 2023B1212060036, the Basic and Applied Basic Research Foundation of Guangdong Province (2025A1515011097), and the Outstanding Youth Project of Guangdong Basic and Applied Basic Research Foundation (2023B1515020064). This work is also supported by Engineering Research Center of Trustworthy AI, Ministry of Education.

References

[1] M. Wang, R. Xu, W. Deng, H. Huang, Implicit face model: depth super-resolution for 3D face recognition, Pattern Recognit. 162 (2025) 111353.
[2] X. Guo, F. Jiang, Q. Chen, Y. Wang, K. Sha, J. Chen, Deep learning-enhanced environment perception for autonomous driving: MDNet with CSP-DarkNet53, Pattern Recognit. 160 (2025) 111174.
[3] S. Deng, X. Zhang, S. Jiang, A diagnostic report supervised deep learning model training strategy for diagnosis of COVID-19, Pattern Recognit. 149 (2024) 110232.
[4] J. Guo, Y. Li, X. Chen, H. Guo, L. Sun, C. Liu, Scale-up: an efficient black-box input-level backdoor detection via analyzing scaled prediction consistency, International Conference on Learning Representations (2023).
[5] B. Mu, Z. Niu, L. Wang, X. Wang, Q. Miao, R. Jin, G. Hua, Progressive backdoor erasing via connecting backdoor and adversarial attacks, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 20495–20503.
[6] Y. Shi, M. Du, X. Wu, Z. Guan, J. Sun, N. Liu, Black-box backdoor defense via zero-shot image purification, Adv. Neural Inf. Process. Syst. 36 (2023) 57336–57366.
[7] W. Tang, J. Li, Y. Rao, Z. Zhou, F. Peng, A trigger-perceivable backdoor attack framework driven by image steganography, Pattern Recognit. 161 (2025) 111262.
[8] T. Gu, K. Liu, B. Dolan-Gavitt, S. Garg, BadNets: evaluating backdooring attacks on deep neural networks, IEEE Access 7 (2019) 47230–47244.
[9] Y. Liu, S. Ma, Y. Aafer, W.-C. Lee, J. Zhai, W. Wang, X. Zhang, Trojaning attack on neural networks, in: 25th Annual Network And Distributed System Security Symposium (NDSS 2018), Internet Soc, 2018.
[10] T.A. Nguyen, A. Tran, Input-aware dynamic backdoor attack, Adv. Neural Inf. Process. Syst. 33 (2020) 3454–3464.
[11] X. Chen, C. Liu, B. Li, K. Lu, D. Song, Targeted backdoor attacks on deep learning systems using data poisoning, arXiv preprint arXiv1712.05526 (2017).
[12] T.A. Nguyen, A.T. Tran, WaNet - Imperceptible warping-based backdoor attack, in: International Conference on Learning Representations, 2021.
[13] Y. Li, Y. Li, B. Wu, L. Li, R. He, S. Lyu, Invisible backdoor attack with sample-specific triggers, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 16463–16472.
[14] J. Zhang, C. Dongdong, Q. Huang, J. Liao, W. Zhang, H. Feng, G. Hua, N. Yu, Poison ink: robust and invisible backdoor attack, IEEE Trans. Image Process. 31 (2022) 5691–5705.
[15] Z. Cheng, B. Wu, Z. Zhang, J. Zhao, Tat: targeted backdoor attacks against visual object tracking, Pattern Recognit. 142 (2023) 109629.
[16] K. Wang, H. Deng, Y. Xu, Z. Liu, Y. Fang, Multi-target label backdoor attacks on graph neural networks, Pattern Recognit. 152 (2024) 110449.
[17] Y. Gao, C. Xu, D. Wang, S. Chen, D.C. Ranasinghe, S. Nepal, Strip: a defence against trojan attacks on deep neural networks, in: Proceedings of the 35th Annual Computer Security Applications Conference, 2019, pp. 113–125.
[18] X. Xu, Q. Wang, H. Li, N. Borisov, C.A. Gunter, B. Li, Detecting ai trojans using meta neural analysis, in: 2021 IEEE Symposium on Security and Privacy (SP), IEEE, 2021, pp. 103–120.
[19] K. Liu, B. Dolan-Gavitt, S. Garg, Fine-pruning: defending against backdooring attacks on deep neural networks, in: International Symposium on Research in Attacks, Intrusions, and Defenses, Springer, 2018, pp. 273–294.
[20] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, B.Y. Zhao, Neural cleanse: identifying and mitigating backdoor attacks in neural networks, in: 2019 IEEE Symposium on Security and Privacy (SP), IEEE, 2019, pp. 707–723.
[21] Z. Wang, K. Mei, J. Zhai, S. Ma, UNICORN: a unified backdoor trigger inversion framework, in: The Eleventh International Conference on Learning Representations, 2023.
[22] Y. Liu, Y. Xie, A. Srivastava, Neural trojans, in: 2017 IEEE International Conference on Computer Design (ICCD), IEEE, 2017, pp. 45–48.
[23] B.G. Doan, E. Abbasnejad, D.C. Ranasinghe, Februus: input purification defense against trojan attacks on deep neural network systems, in: Annual Computer Security Applications Conference, 2020, pp. 897–912.
[24] R.R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra, Grad-cam: visual explanations from deep networks via gradient-based localization, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 618–626.
[25] W. Xu, D. Evans, Y. Qi, Feature squeezing: detecting adversarial examples in deep neural networks, In Network and Distributed System Security (NDSS) Symposium (2018).
[26] Y. Li, T. Zhai, Y. Jiang, Z. Li, S.-T. Xia, Backdoor attack in the physical world, ICLR 2021 workshop on RobustML (2021).
[27] T. Sun, L. Pang, C. Chen, H. Ling, Mask and restore: blind backdoor defense at test time with masked autoencoder, arXiv preprint arXiv:2303.15564 (2023).
[28] I.J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, Stat 1050 (2015) 20.
[29] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, in: International Conference on Learning Representations, 2018.
[30] D. Wang, W. Yao, T. Jiang, C. Li, X. Chen, Rfla: a stealthy reflected light adversarial attack in the physical world, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 4455–4465.
[31] Y. Bai, Y. Wang, Y. Zeng, Y. Jiang, S.-T. Xia, Query efficient black-box adversarial attack on deep neural networks, Pattern Recognit. 133 (2023) 109037.
[32] O. Oktay, J. Schlemper, L.L. Folgoc, M. Lee, M. Heinrich, K. Misawa, K. Mori, S. McDonagh, N.Y. Hammerla, B. Kainz, B. Glocker, D. Rueckert, Attention U-Net: learning where to look for the pancreas, in: Medical Imaging with Deep Learning, 2018.
[33] S. Paris, A gentle introduction to bilateral filtering and its applications, in: ACM SIGGRAPH 2007 courses, 2007, pp. 3–es.
[34] A. Krizhevsky, G. Hinton, Learning multiple layers of features from tiny images, Handbook of Sys. Autoimmun. Dis. 1 (4) (2009).

[35] J. Stallkamp, M. Schlipsing, J. Salmen, C. Igel, The German traffic sign recognition benchmark: a multi-class classification competition, in: The 2011 International Joint Conference on Neural Networks, IEEE, 2011, pp. 1453–1460.
[36] J. Howard, Imagenette - a subset of 10 easily classified classes from the imagenet dataset, 2019, https://github.com/fastai/imagenette.
[37] B. Wu, H. Chen, M. Zhang, Z. Zhu, S. Wei, D. Yuan, C. Shen, Backdoorbench: a comprehensive benchmark of backdoor learning, Adv. Neural Inf. Process. Syst. 35 (2022) 10546–10559.
[38] K. He, X. Zhang, S. Ren, J. Sun, Identity mappings in deep residual networks, in: Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, the Netherlands, October 11–14, 2016, Proceedings, Part IV 14, Springer, 2016, pp. 630–645.
[39] J.C. Neves, R. Tolosana, R. Vera-Rodriguez, V. Lopes, H. Proença, J. Fierrez, Ganprintr: improved fakes and evaluation of the state of the art in face manipulation detection, IEEE J. Sel. Top. Signal Process. 14 (5) (2020) 1038–1048.
[40] A. Turner, D. Tsipras, A. Madry, Clean-label backdoor attacks, International Conference on Learning Representations (2018).